

RAD STUDIO 13.1 Florence



Launch Webinar



Recap of RAD Studio 13.0

Top New Features in 13.0 Florence

- Delphi language: Ternary *if* operator
- Clang 20 upgrade for C++Builder
- Complete 64-bit IDE for Windows 64-bit platform
- VA & code tooling improvements
- SmartCore AI Component Pack
- VCL TitleBar Styling and new FMX controls
- WebBroker and WebStencils extensions



Delphi Compiler for Windows on Arm

Target the next generation of Windows on Arm devices with your existing codebase

- New native target platform for Delphi compiler: Windows on Arm
- Delphi now supports creating native Arm binaries that run on Windows Arm devices, without using the Intel emulation layer
- Allows developers to expand the Windows support, building native applications for Win32 Intel, Win64 Intel and Win64 Arm using the same source code
- Existing Delphi applications can generally be recompiled for the new target platform.



Windows on Arm and Arm64EC

- What is Arm64EC?

“Arm64EC (Emulation Compatible) is a Microsoft Application Binary Interface (ABI) for Windows on Arm that allows native Arm64 code to interoperate seamlessly with emulated x64 code, enabling incremental migration by sharing x64 calling conventions and registers”

- Windows on Arm can run

- Arm64: Native Windows on Arm applications
 - Can only load Arm64 modules (DLLs, drivers, etc)
- X64: Windows Intel x64 applications (using an emulation engine)
 - Can only load Intel x64 modules (DLLs, drivers, etc)
- Arm64EC: Native Windows on Arm applications
 - Can load both Arm64EC and Intel x64 modules (using emulation)

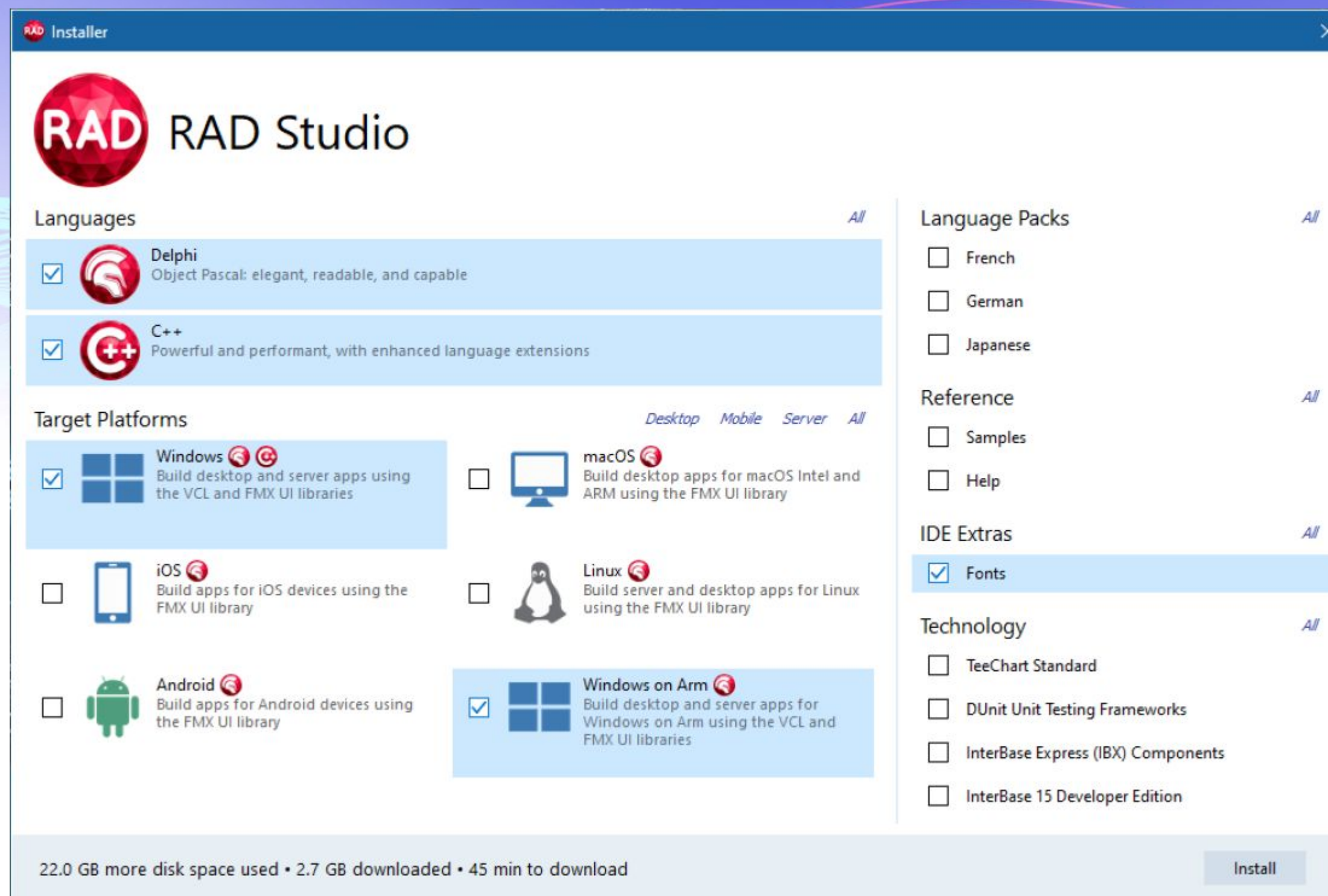


New Delphi Target Platform: Arm64EC

- Delphi Arm64EC compiler is LLVM based
 - We moved to LLVM version 20 (not for other Delphi LLVM compilers)
- New Arm64EC platform toolchain includes:
 - The full Delphi language spec
 - The entire RTL, VCL, FMX, FireDAC, Internet libraries, and much more
- Planned for a future release:
 - Ability to create dynamic packages for Arm (RAD Server is not supported in 13.1, as it relies on dynamic packages)

Minimum OS version for Arm is Windows 11 and Windows Server 2025

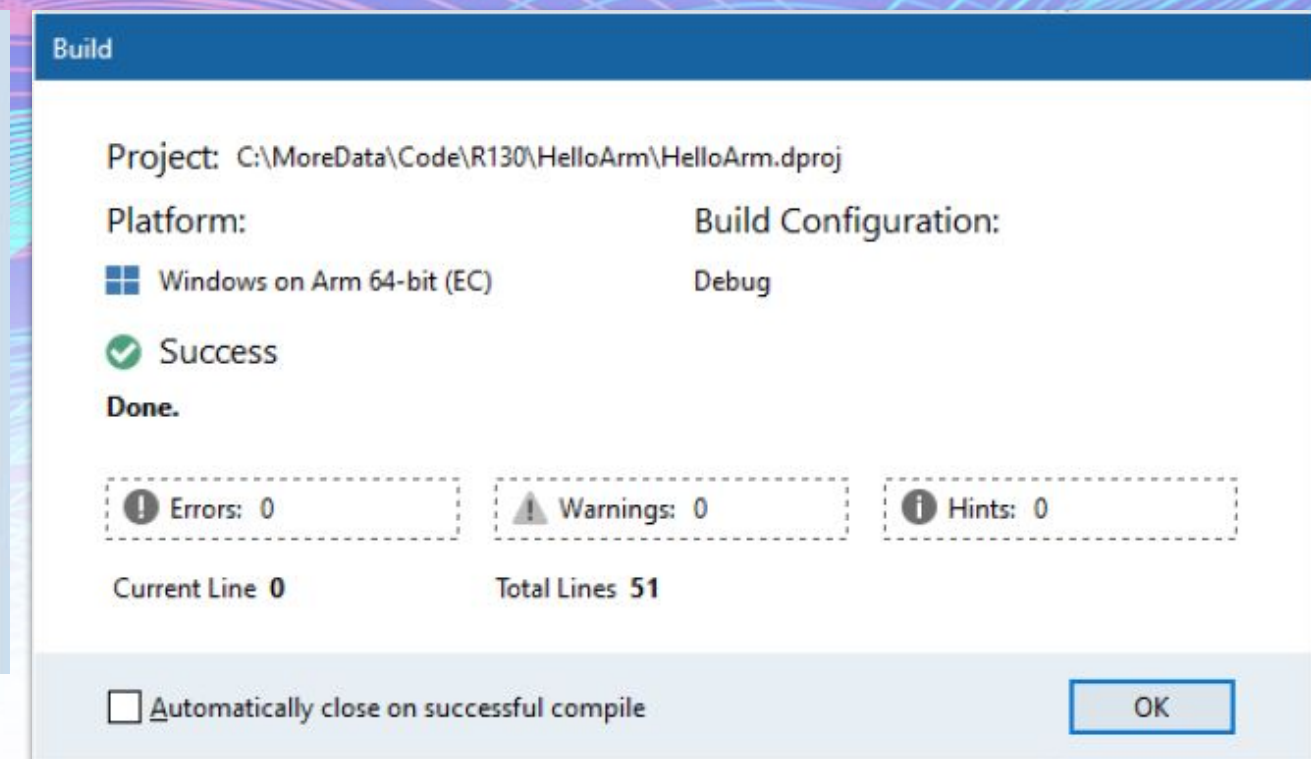
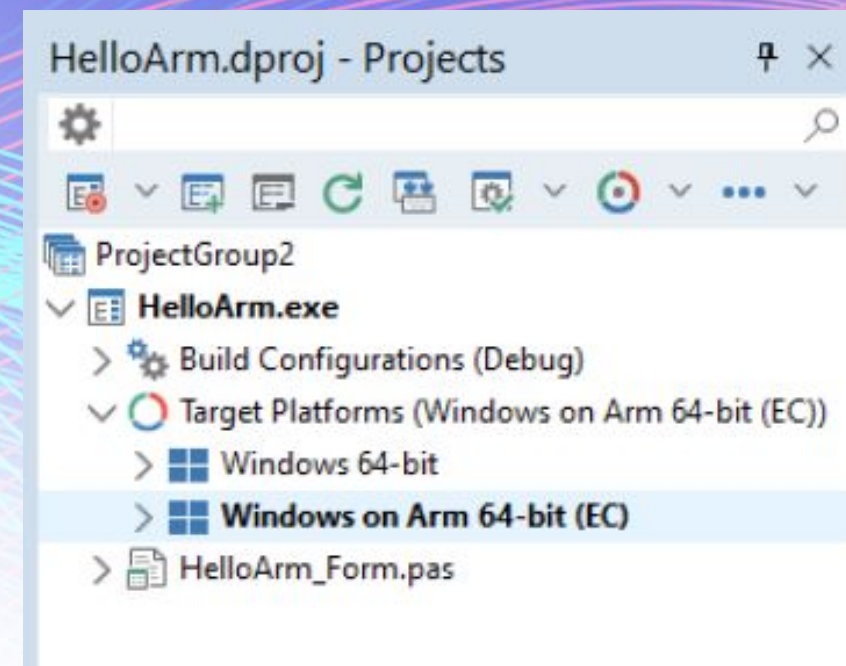
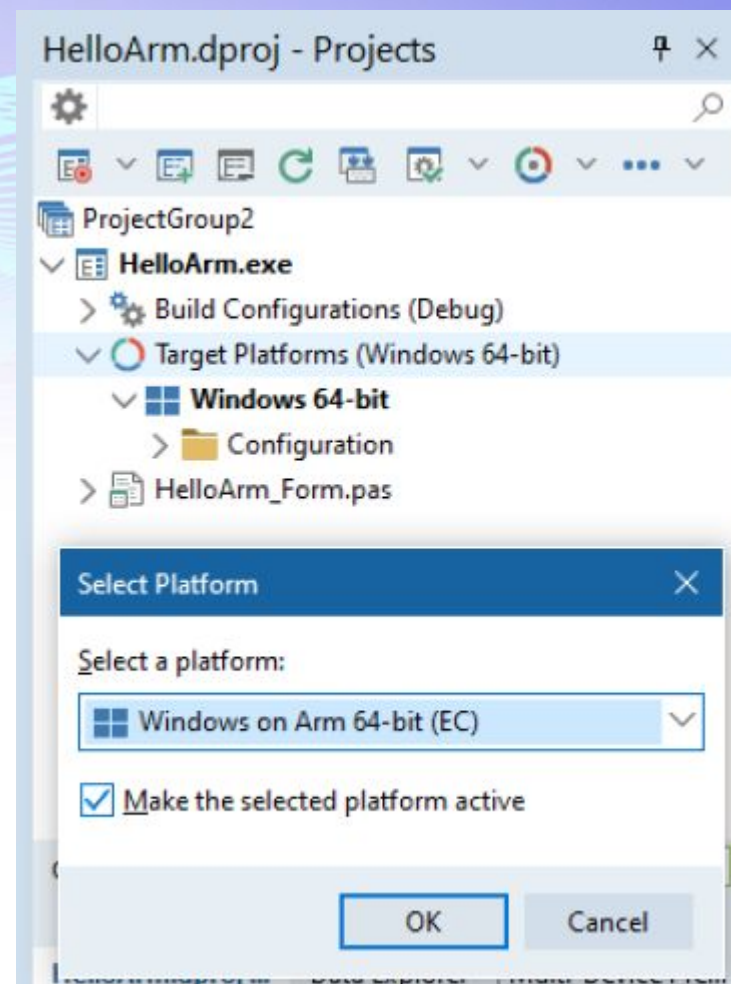
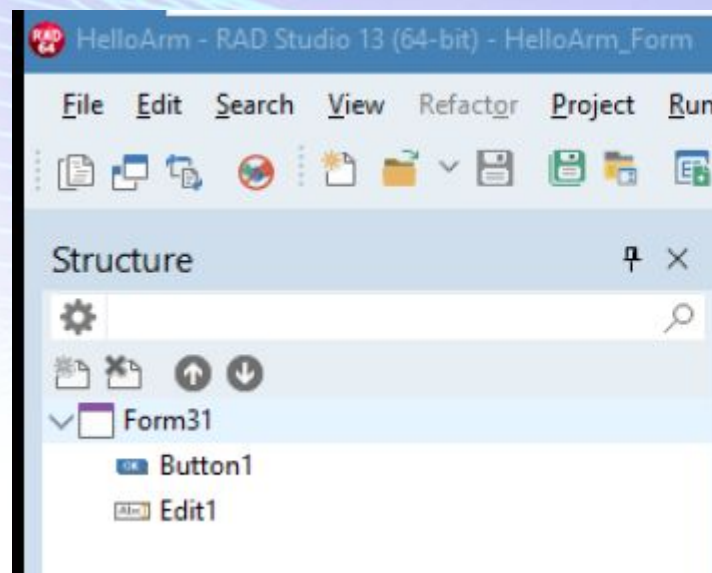
New Platform in Manage Platforms dialog



New Target Platform in 64-bit IDE only



64-bit IDE -> Add platform -> Activate -> Build





Native Arm64 (x64 Compatible) Exe

- Task manager on Windows on Arm shows the format of running executables. The 3 below are the same Delphi app built for:
 - Intel Win32 (x86) emulated
 - Intel Win64 (x64) emulated
 - Arm64 native (in the Arm64EC “x64 compatible” flavor)

 fontdrvhost.exe	8836	Running	UMFD-2	00	1,400 K	Arm64
 HelloArm.exe	6576	Running	MarcoCantu	01	6,144 K	x86
 HelloArm.exe	14096	Running	MarcoCantu	00	7,064 K	x64
 HelloArm.exe	12520	Running	MarcoCantu	00	5,644 K	Arm64 (x64 compatible)

darkdash - RAD Studio 13 (64-bit) - modules.dm1 [Built]

File Edit Search View Refactor Project Run Component Tools Tabs Help

Structure

forms.mainform modules.dm1

Search for a type

Structure

interface

implementation

var

DataModule1: TD

implementation

uses FMX.Dialogs;

{%CLASSGROUP 'FMX

{ \$R *.dfm }

const

////

//

// Change these

//

cDatabase = 'c

cServer =

cPort =

cUser_Name = 'S

cPassword = 'P

////

Dark Dash

Hello Ian

Select period: From Jan 1, 2025 to Dec 31, 2025

New Automation

New Template

Filter

Search...

Total Income

\$250,000

Inquiry Success Rate

36.2%

Number of New Clients

217

Number of Completed Projects

34

Inquiry Breakdown

Website

Email

Direct

Partner

Reseller

Social

Inquiries per Month

Inquiry Source Breakdown

Inquiry Month

Income Source per Month

Task Manager

Type a name, publisher, or PID to sea...

Details

Run new task End task

Name	PID	Status	User name	CPU	Worki...	Platform	Architecture	UAC virtualizati...
darkdash.exe	10384	Running	ianbarker	01	0 K	64 bit	Arm64 (x64 compatible)	Disabled
svchost.exe	12640	Running	SYSTEM	00	0 K	64 bit	Arm64	Not allowed
svchost.exe	10476	Running	SYSTEM	00	0 K	64 bit	Ar	
smartscreen.exe	12584	Running	ianbarker	00	0 K	64 bit	Ar	
audiodg.exe	6720	Running	LOCAL SERVICE	00	0 K	64 bit	Ar	
prevhost.exe	2064	Running	ianbarker	00	0 K	64 bit	Ar	
winlogon.exe	824	Running	SYSTEM	00	0 K	64 bit	Ar	
svchost.exe	616	Running	SYSTEM	00	104 K	64 bit	Ar	
WUDFHost.exe	640	Running	LOCAL SERVICE	00	0 K	64 bit	Arm64	Not allowed
fontdrvhost.exe	552	Running	UMFD-1	00	0 K	64 bit	Arm64	Disabled
fontdrvhost.exe	620	Running	UMFD-0	00	0 K	64 bit	Arm64	Disabled

55: 40

Insert

Windows ARM 64-bit

7:08 PM 2/26/2026

More on Delphi Arm64EC



- Available in 64-bit IDE only (not the 32-bit IDE)
- Debugger uses LLDB for Windows on Arm
 - Can debug on a remote Windows on Arm machine via PA Server or locally if RAD Studio is installed on a Windows on Arm device
- Migration suggestions
 - Arm64EC compiler does not support inline assembly or Intel assembly
 - Use ARM64EC compiler define if needed
 - But also MSWINDOWS and CPUARM64



Arm64EC and Windows API

- The largest majority of the Windows APIs are fully compatible
- VirtualAlloc will return pages for x64 instructions
 - Calls should be moved to VirtualAlloc2 with MEM_EXTENDED_PARAMETER_EC_CODE
- HeapCreate/HeapAlloc cannot be used for runtime code injection
 - The resulting pages can only be used to execute x64 instructions
- GetNativeSystemInfo returns emulated CPU on Arm64EC
 - TOSVersion.Architecture calls it, has the same design
 - Use IsWow64Process2 for actual CPU detection

Linking External Object Files on Arm64EC



- WinARM64EC compiler does **not** support merging external object files using the {\$L 'foo.o'} syntax
- Declare the external symbol on the function declaration using *external object* syntax to link individual object files:

```
function zlibVersion: MarshaledAString; cdecl;  
external object 'zutil.o';
```

- Recommended alternative is to package object files into a static library (.a) and link against that library instead

```
function zlibVersion: MarshaledAString; cdecl; external 'libzlib.a';
```

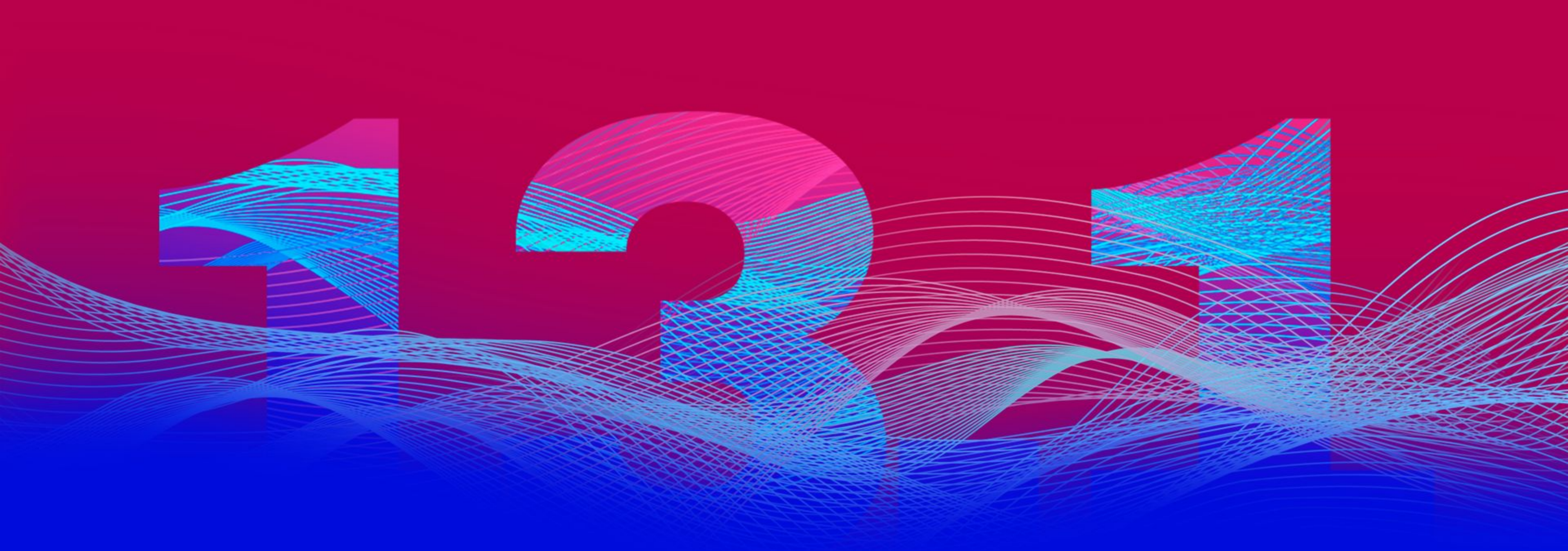



DLL compatibility in Arm64EC

- Arm64EC enables a hybrid execution model in which a single process can contain both native 64 code and x64 (AMD64) code
- Delphi applications targeting Arm64EC can load and use x64 DLLs without restrictions

```
procedure Bar; external 'foo.dll';
```

- Arm64EC applications can also load DLLs built for Arm64EC
- However, pure Arm64 DLLs are not compatible with Arm64EC processes



Compilers and Platforms

C++Builder Quality



- Primarily focused on improving quality for C++Builder 64-bit Modern compiler toolchain (upgraded to Clang 20 and C++23 in 13.0)
- Added the clang-scan-deps utility for faster dependency scanning
- Enhanced Unicode path handling for linker invocations
- Several issues related to C++ use of the Delphi runtime have been resolved
- Windows SDK import improvements

Changes to the Windows SDK integration



- This release improves the algorithm for importing Windows SDK
 - The algorithm no longer depends on the order of SDK installation
 - Installation of unsupported Windows SDK alongside supported versions no longer leads to errors during import
 - More reliable when multiple versions of SDK are installed
- 13.1 adds support for the most recent Windows SDK



Delphi Compiler Quality

- Fixed a regression with some ambiguous calls and type incompatibilities when using AnsiChar and ShortString
 - For example, when calling Pos
- Improved floating-point and integer type compatibility when using the new ternary if operator
- New warning message triggered when using noreturn for a procedure or function with a return path
 - “W1080 NORETURN procedure %s should not return”
 - Matching {\$WARN INVALID_NORETURN ON/OFF/ERROR/DEFAULT}

Android API level 36



- RAD Studio 13.1 adds support for Android API level 36.1
 - Mandatory requirement for publishing Play Store starting August 2026
 - SDK installation uses the 'Platform 36.1' Android SDK package
 - Upgraded build system and platform jar files for Android API level 36.1
 - Updated the Application Uses Permissions options page to include the new permissions required for API level 36
 - Upgraded the version of the JetPack Core library
 - Support the “opt out of predictive back navigation” feature on Android 16

Warning: This requires a manifest change. If you are updating an existing 13.0 installation, you need to clear the '%APPDATA%\Embarcadero\BDS\37.0' directory to avoid using the 13.0 version of the manifest in new applications



More for the Android Toolchain

- 13.1 enables building a so-library for Android
 - Delphi Android compilers support passing the '-h' option to the external linker to generate '.so' files with the expected 'DT_SONAME' entry in the '.dynamic' section
- Automatic detection of the Android SDK path, in order:
 - 'ANDROID_HOME' environment variable
 - Old 'ANDROID_SDK_ROOT' environment variable
 - Detection based on the 'AndroidSDK-NDK' GetIt package
 - Default installation directory used by Android Studio the '%LOCALAPPDATA%\Android\Sdk' directory



Official Support for iOS 26

- RAD Studio is adding official support for iOS 26
- Increased default value of the Minimum iOS version supported to 15.0
 - *Overall goal is your applications remain compatible with Apple iOS and App Store requirements*
- Migrated to the UIKit scene-based lifecycle
 - From the deprecated UIScene lifecycle
 - Changes include the info.plist configuration file (which needs updating)
 - Significant impact to library code. Required redesign of modal dialogs, among other changes

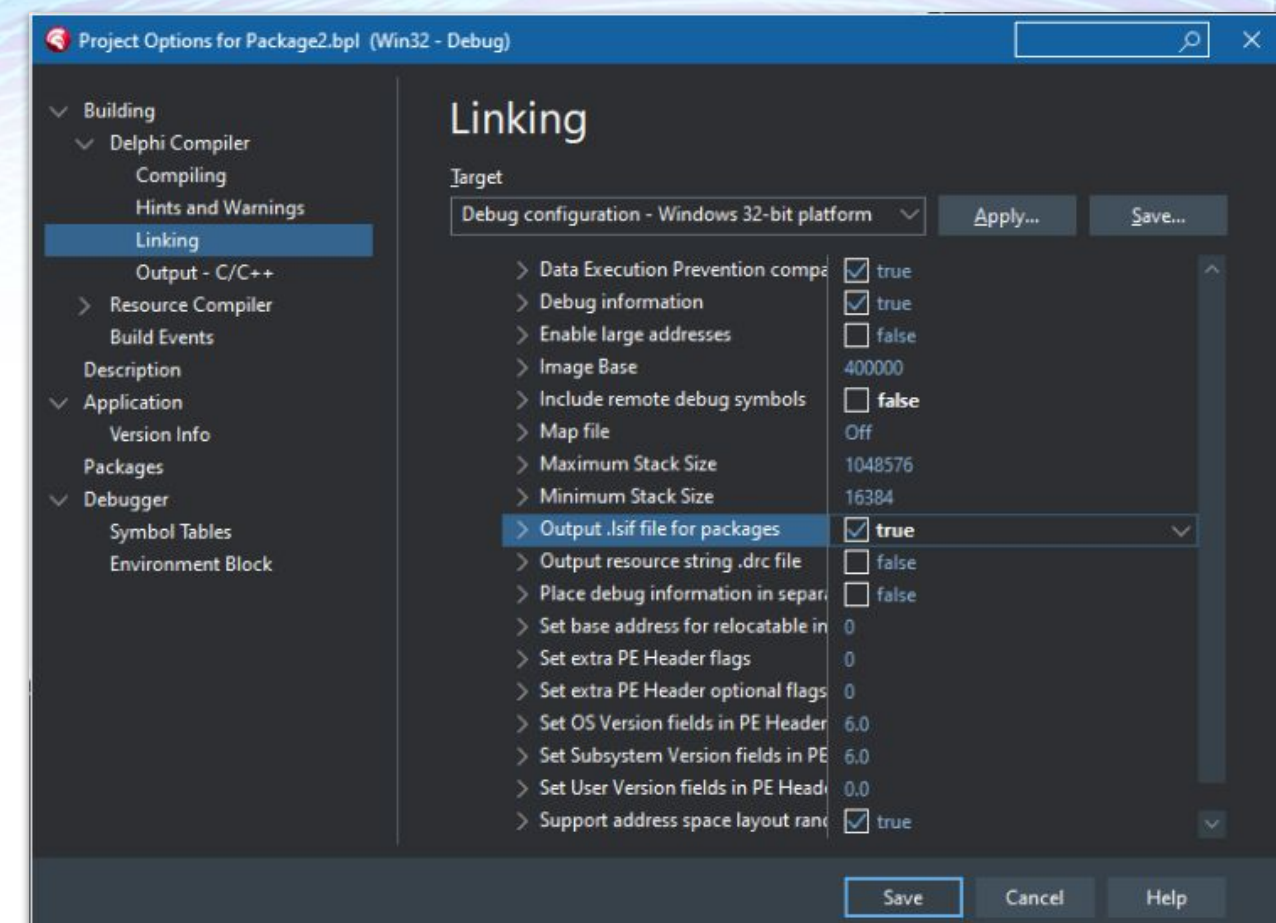


RAD Studio IDE

DelphiLSP LSIF



- Introducing support for Language Server Index Format (LSIF) in DelphiLSP architecture
 - Aims to reduce dependence on the compiler, improving the performance, stability, and accuracy of Code Insight
- When building a package you can generate an LSIF file
- When IDE uses units from the package it loads the LSIF file rather compiling
 - Used for **navigation** requests
- Can be turned off using a registry setting



DelphiLSP LSIF

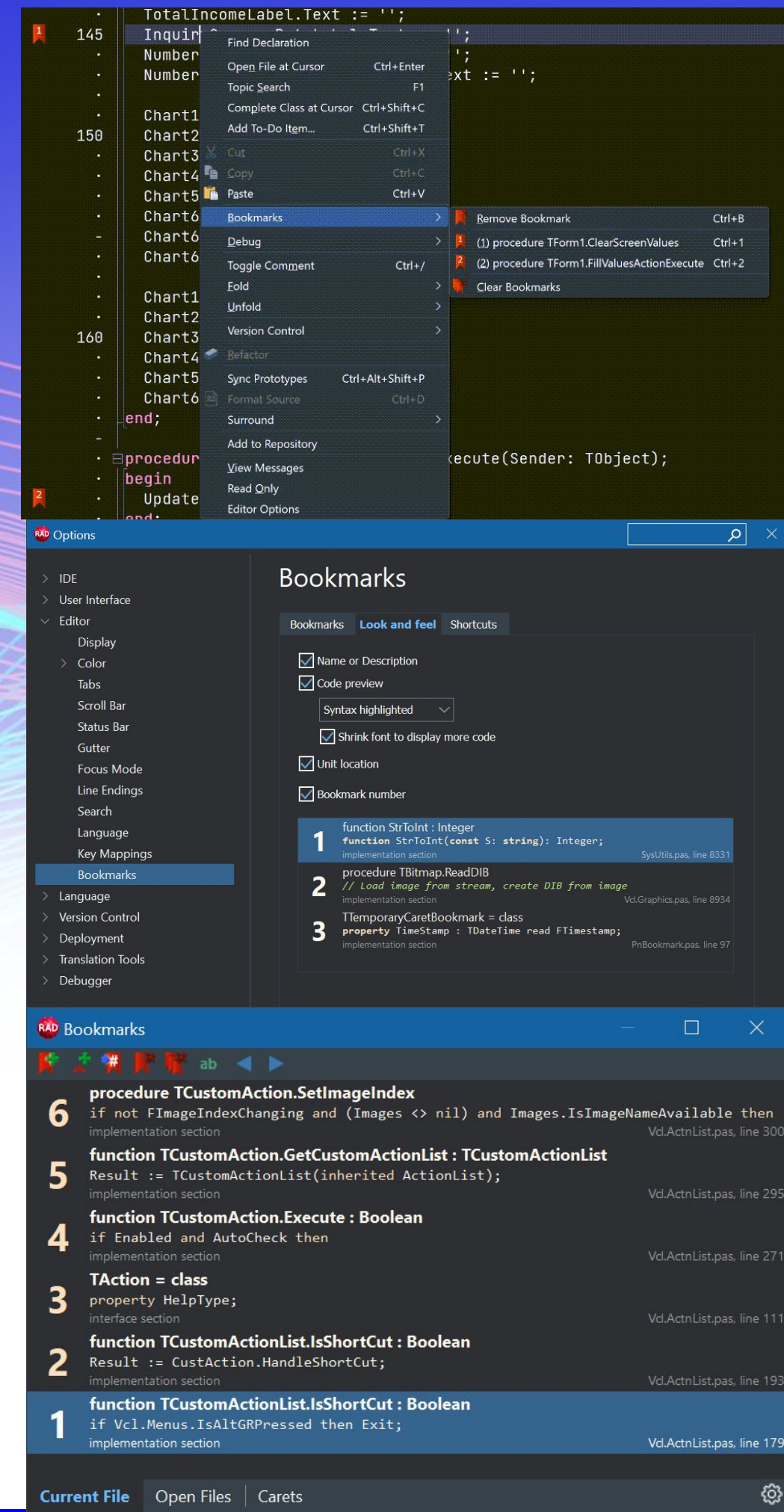


- Clarification: The LSIF caching works when looking at symbols in the source code of units part of packages which include LSIF
 - Not in units using symbols defined in those packages
 - In other words, if I click on TForm in my unit1, it goes via compiler; if I click on TForm in Forms.pas or another core VCL unit, it goes via LSIF
- For this feature to offer significant benefit it's very important that our customers use packages with LSIF generation
 - This is why we want to ask all third party vendors to add LSIF generation to their packages, to help customers have a more stable DelphiLSP engine when using their components

Bookmarks plugin integration

Expanded built-in source bookmarks

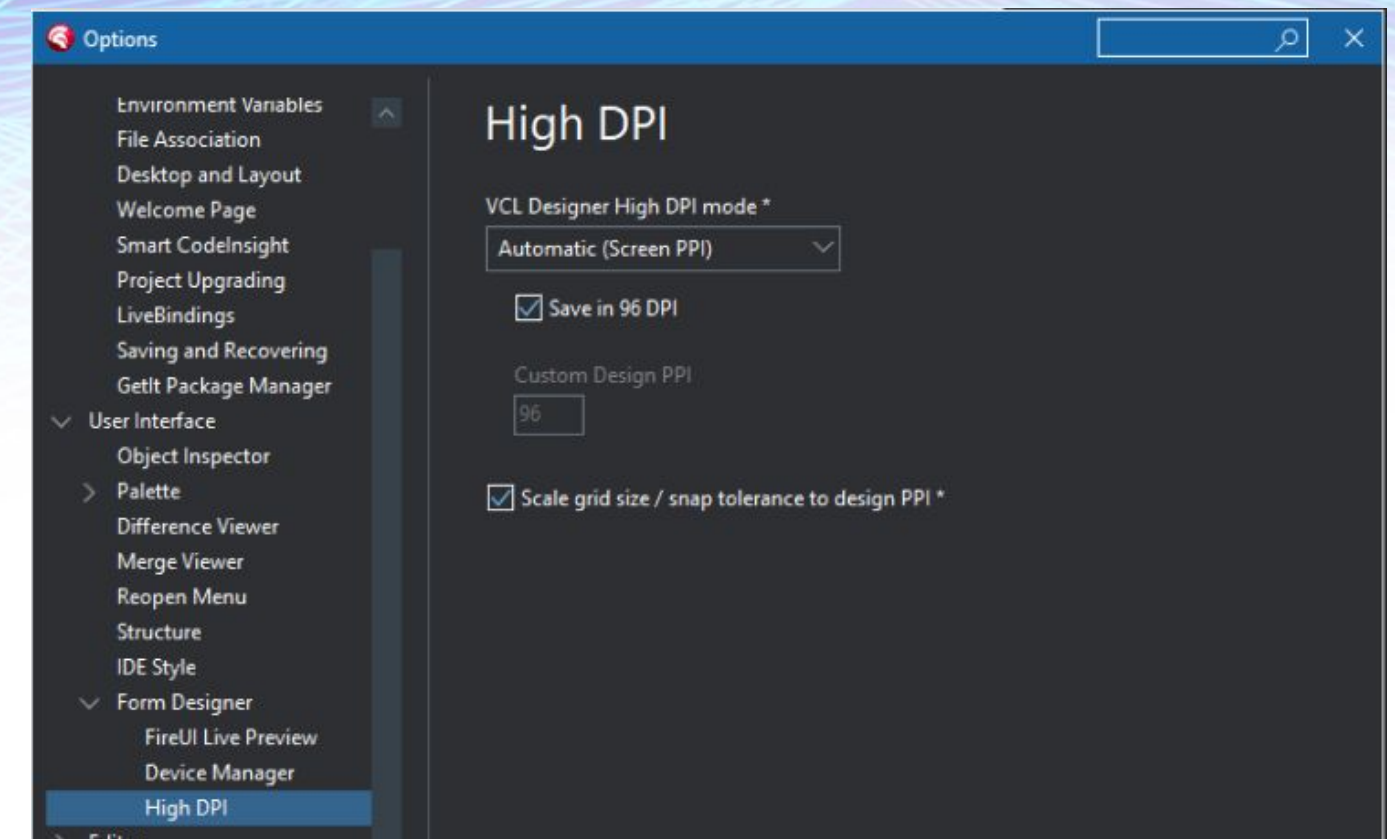
- Full IDE integration of the Bookmarks Addon
 - Originally from Parnassus and previously available as a download in GetIt
- Replaces the older IDE bookmarks
 - Superset of features
- Includes Caret Bookmark feature
 - Use Ctrl+Shift+B to drop a temporary caret bookmark, press Escape to jump back to it





Save VCL DFM files at 96 DPI

- New option to save forms at 96 DPI regardless of the design DPI
 - Particularly handy if you have team members working on the same project with different resolutions and different HighDPI IDE configurations
- Option is available in the Form Designer | High DPI section of the Tools Options

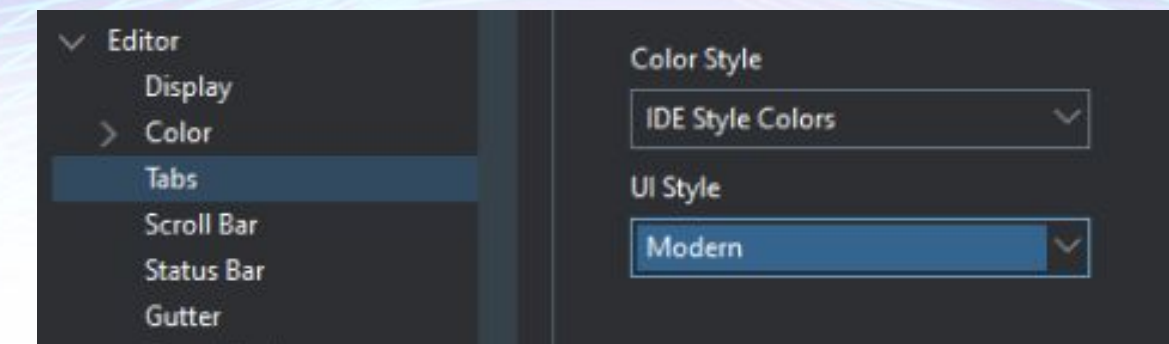




Updated IDE Styles for Windows 11

Modernized look and feel of the IDE

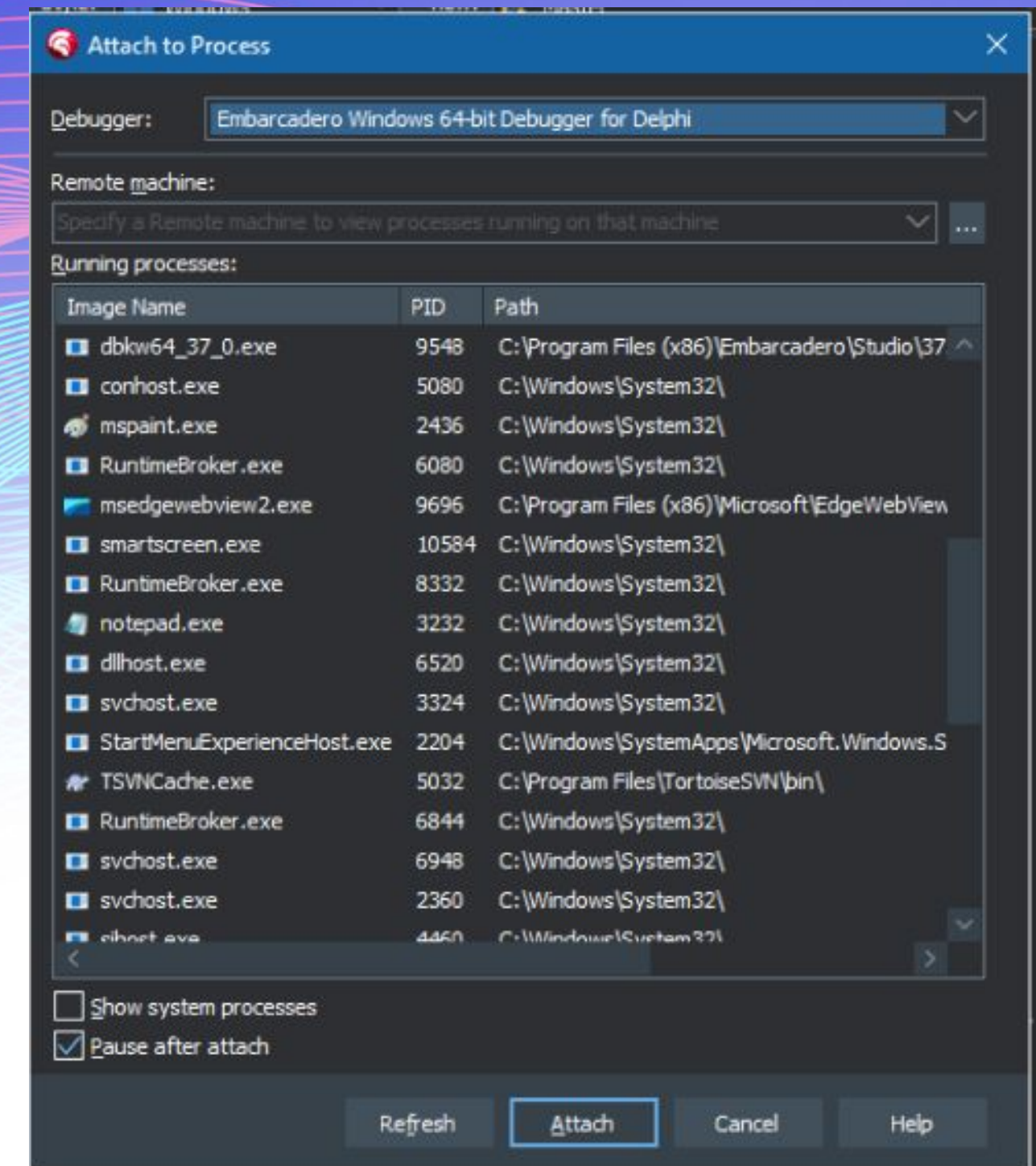
- Updated IDE Styles and UI for Windows 11
 - Light, dark, mountain mist styles
 - Win11 style loaded if you are running IDE under Win11, otherwise falls back to Windows 10 styles
 - Rounded edges in many windows
 - For editor tabs round edges, a specific setting





Attach to Process Dialog

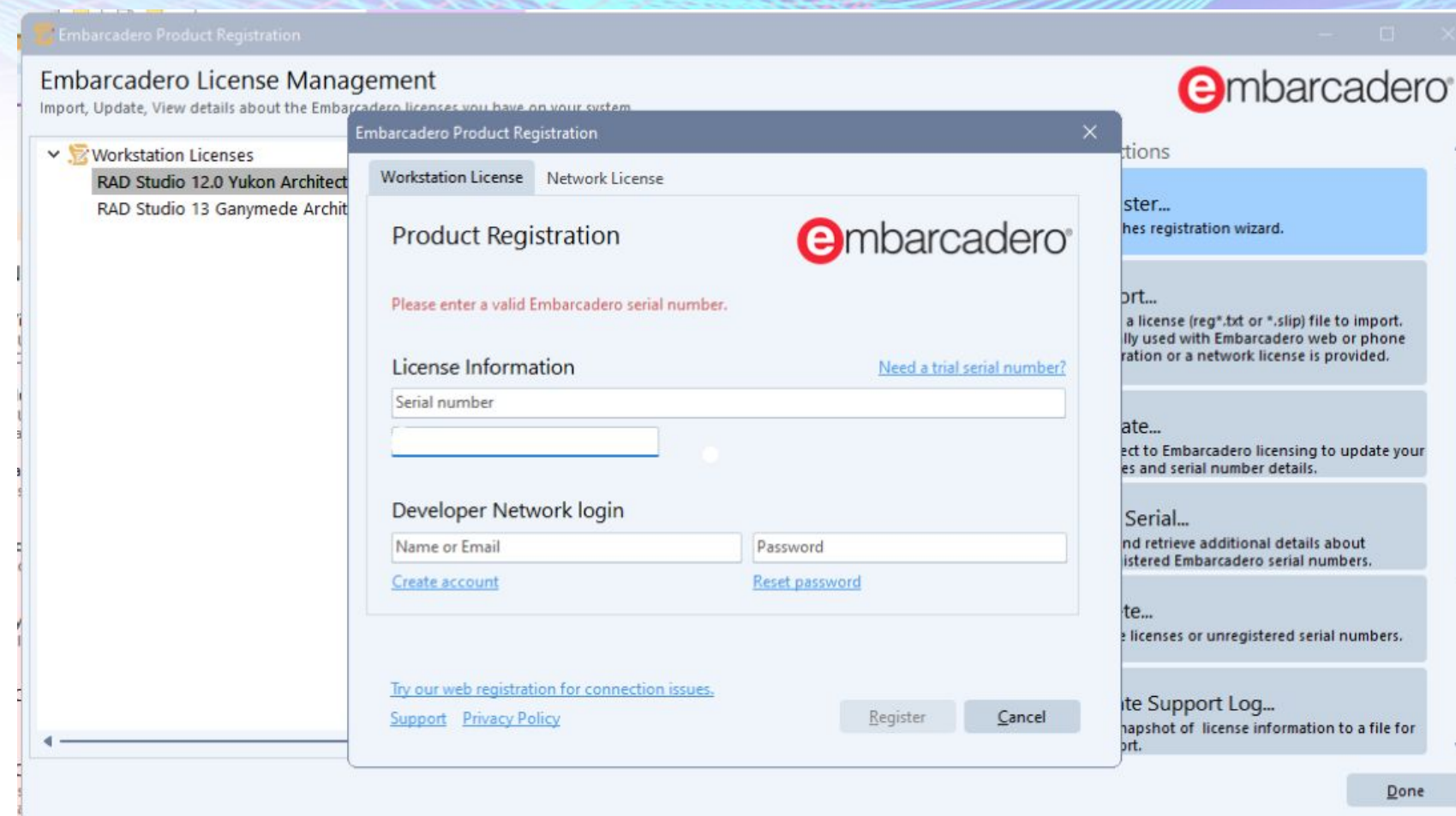
- Attach to Process dialog displays process icons





Product Registration UI Refresh

- More external tools are HighDPI aware
- Product registration UI modernization, updated icons, HighDPI enablement and VCL styles support



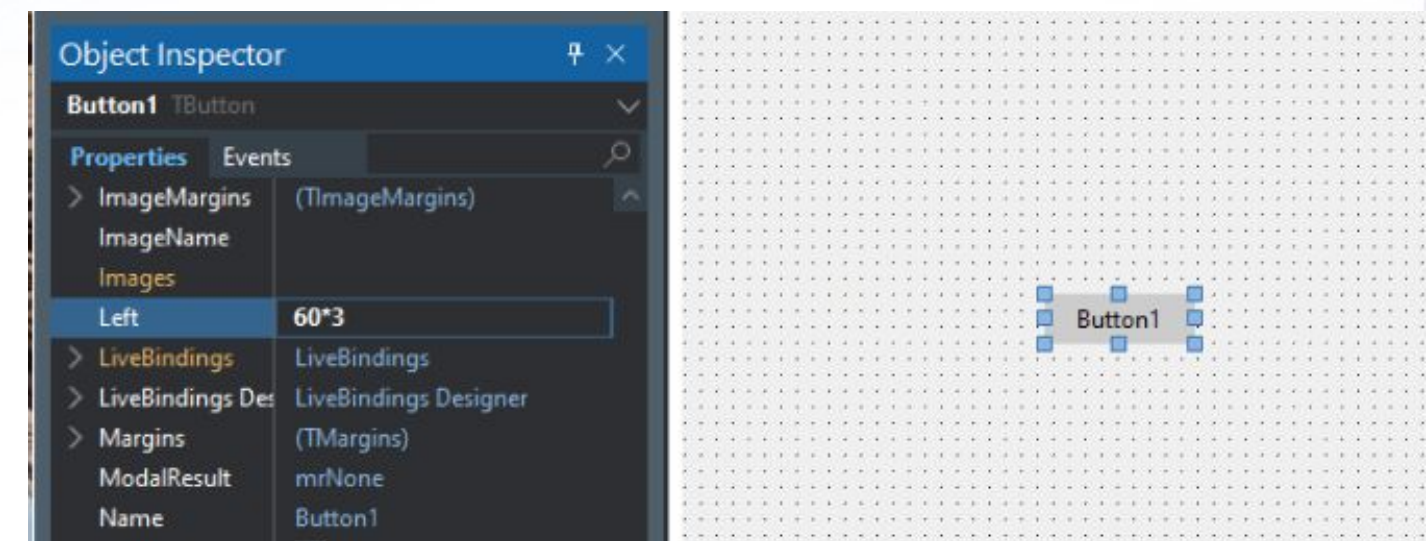
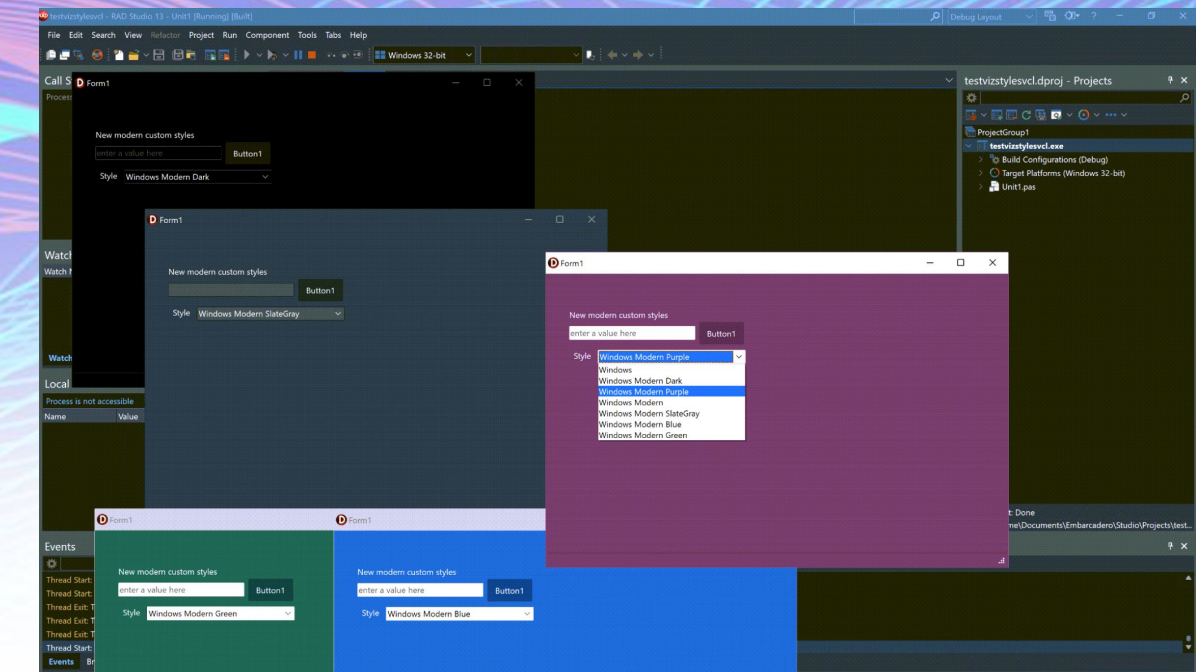


Libraries

VCL Improvements



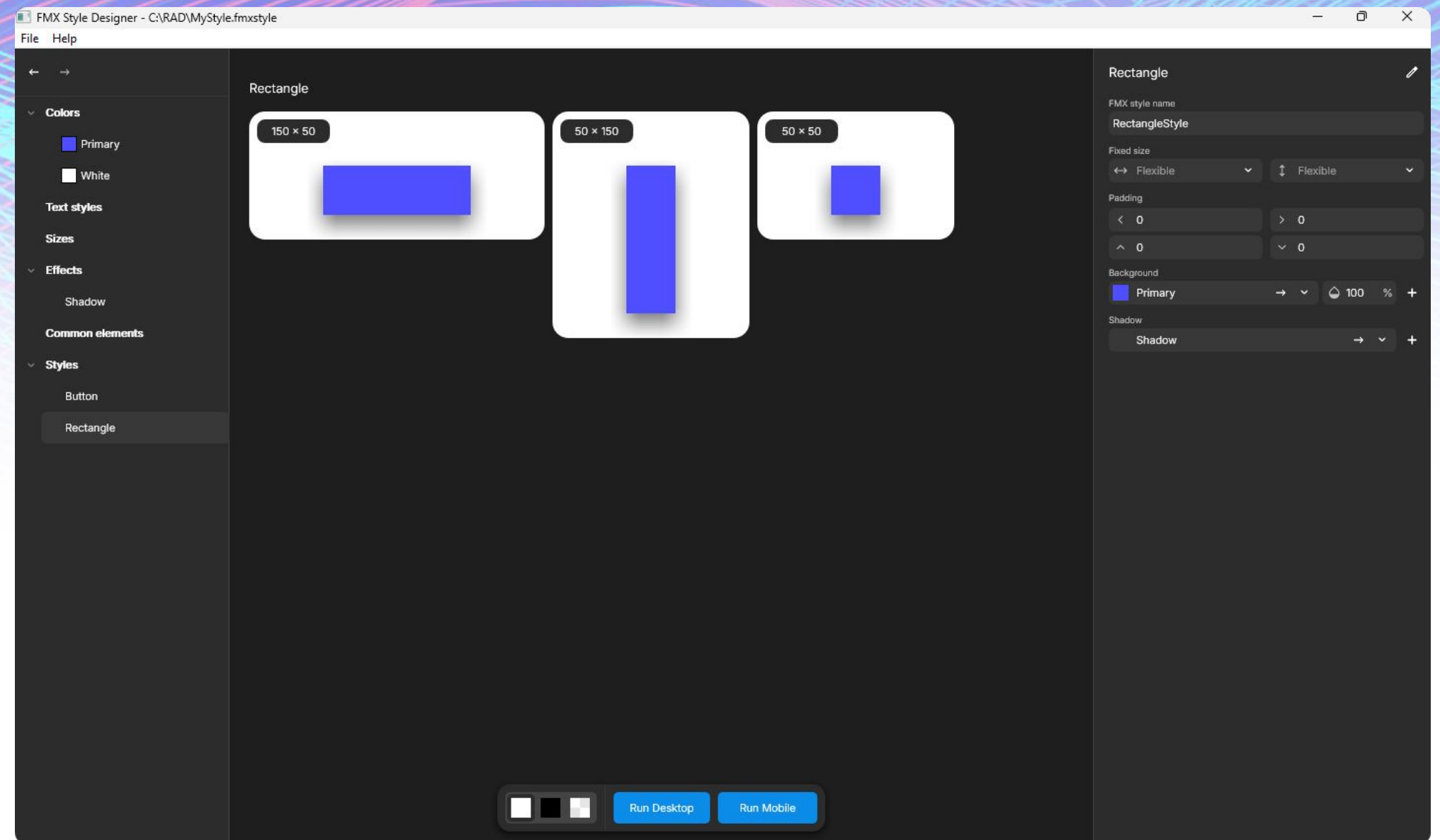
- Easily modernize the look and feel of your Windows applications with new Windows 11 specific styles
 - Windows Modern
 - Windows Modern Dark
 - Windows Modern Blue
 - Windows Modern SlateGray
 - Windows Modern Green
 - Windows Modern Purple
- Object Inspector Expressions
 - Simple calculations allowed in specific numeric properties: Left, Top, Height, and Width properties of TControl
 - *We'll expand based on feedback*



Initial Release of FireMonkey Style Designer

Specifically intended for FMX (unlike the existing Bitmap Style Designer) and based on modern design principles

Standalone application that reimagines how developers create and manage FMX styles for RAD Studio





FireMonkey Style Designer Goals

Faster, more flexible and modern UI styling

- Design-centric interface where you work with high-level concepts: colors, backgrounds, typography, and interaction states
- The tool automatically handles technical translation into FireMonkey's style format.
- Built with FireMonkey, showcasing the framework's capabilities for building professional desktop tooling
- Meant to be used by developers, but also by graphic UI designers working alongside the development team

FireMonkey Style Designer: Core Concepts

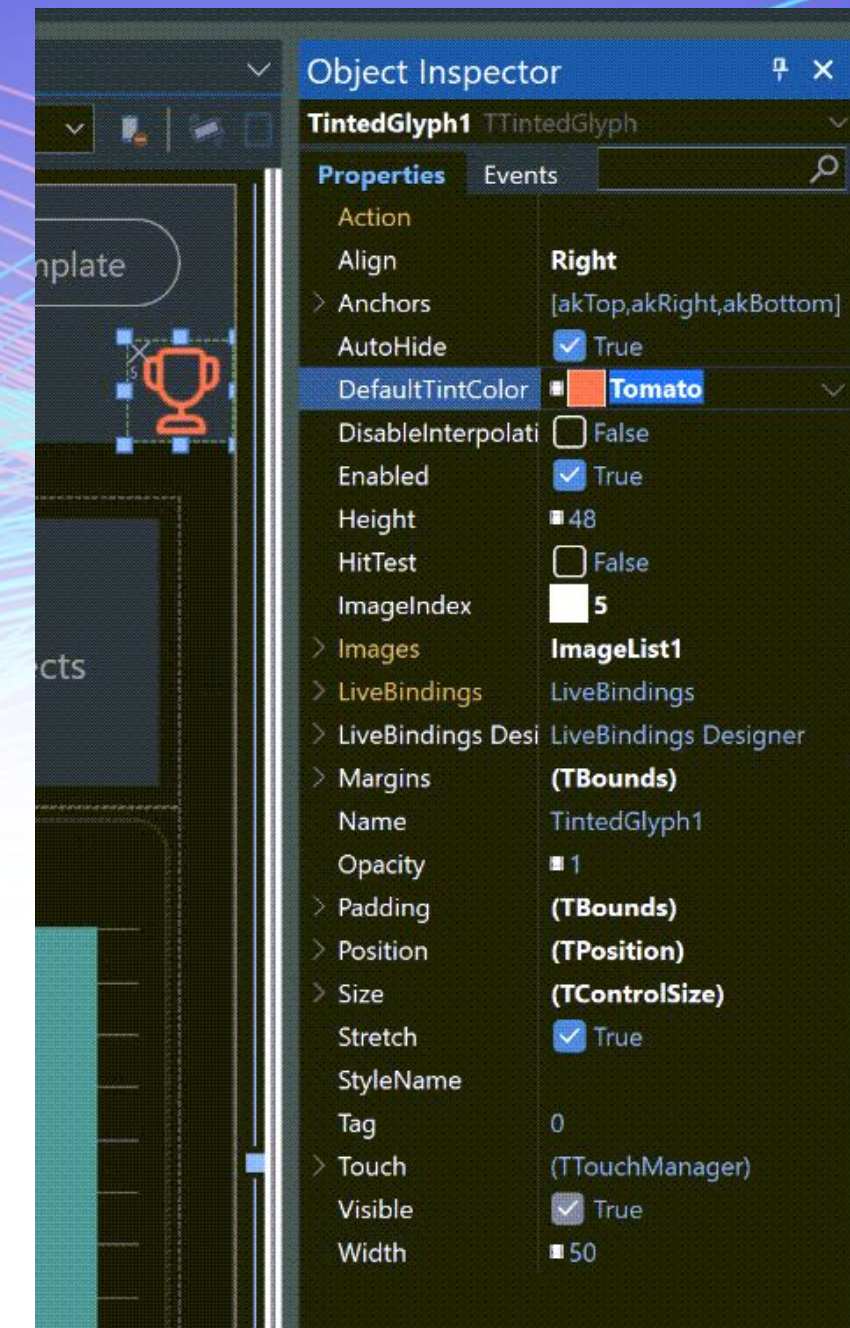


- Parameters (Design Tokens)
 - Named values referenced throughout your style. Define your brand colors, typography settings, or standard dimensions and reference them in multiple styles
- Resources (Visual Building Blocks)
 - Reusable visual elements: solid colors, vector rectangles, bitmap images, and state-aware variants. Resources define complete appearances
- Hierarchical Organization with Naming Conventions
 - The designer uses hyphen-separated naming to automatically create folder hierarchies for organization
- State-Based Design
 - Interactive components respond to user actions through states: normal, hovered, pressed, focused, selected. Designer makes this explicit via dynamic resources.



Additional FireMonkey Improvements

- New TTintedGlyph control, supports recoloring icons in an ImageList (like TTintedStyleObject for TSpeedButton)
- Skia for Delphi updated to version 7.1.0
- Added support for DirectX texture filtering
- Support for Linux styles and views in the designer (FMXLinux remains a separate installation on GetIt)
- Improved Frame alignments and margins





FireDAC Database Access

- The FireDAC database library is updated regularly to support new database releases
 - SAP ASE Server version 16.1
 - IBM DB2 12.1
 - MariaDB Server 12.1
 - *(some database drivers are available in the Enterprise version only)*
- Legacy SQLite encryption for 64-bit Windows



Internet, HTTP, and REST

- Support for Server-Sent Events (SSE)
 - Server side: TWebResponseStream class, new methods for TWebResponse (BeginStream and BeginEventsStream)
 - Client side: New unit System.Net.HttpSse.pas including THTTPEvent and THTTPEventSource
- Additional Features
 - TWinHTTPClient enable IPv6 with fast fallback to IPv4
 - Apache configuration support (for Delphi in 13.0) added to C++ Builder
 - RAD Server Lite now offers integrated support for TLS 1.3
OpenSSL 3 library not included



Quality Overall



Quality, Quality, quality

- There is a strong push for quality, with extensive focus on:
 - Modern C++ compiler
 - LLDB debuggers for Delphi and C++
 - Runtime libraries
 - Many IDE areas
- Volume of Quality Reports Resolved
 - Around 60 support cases
 - Over 250 public customer reports
 - Addressed almost 1,000 internal tickets

Summary



Top Themes

- Delphi for Windows on Arm
- DelphiLSP LSIF support
- Save as 96 dpi, and new VCL Windows 11 styles
- Improved Windows SDK integration
- Bookmarks plugin in the IDE
- Android API level 36
- Plus quality and many smaller enhancements



Thank You!